

LAB EXAM-4

P.LEELA PRASEEDA SAI

2403A52056

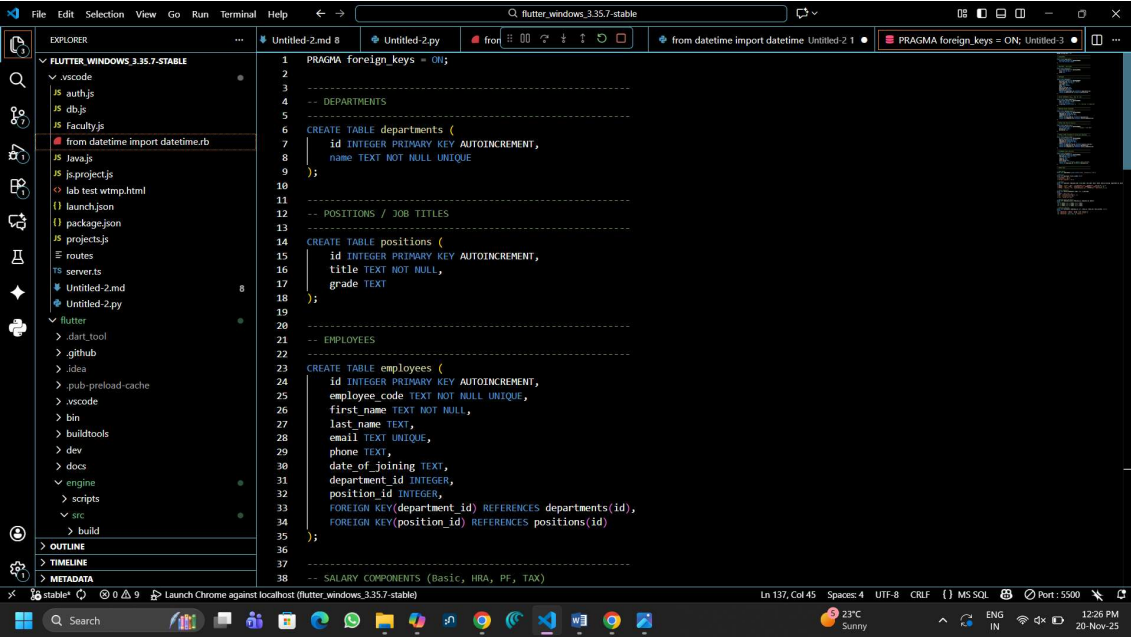
BATCH-03

SET-13

Q1-A) PROMPT:

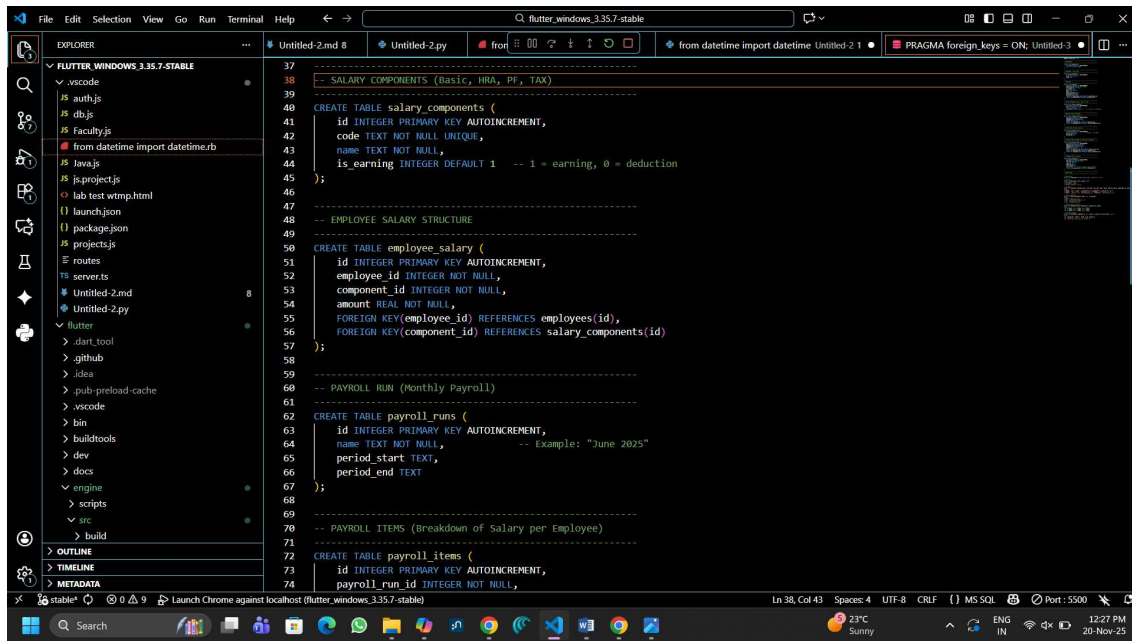
WRITE A CODE TO DESIGN A DATABASE FOR EMPLOYEE PAYROLL AND ATTENDANCE IN SQL.

CODE:



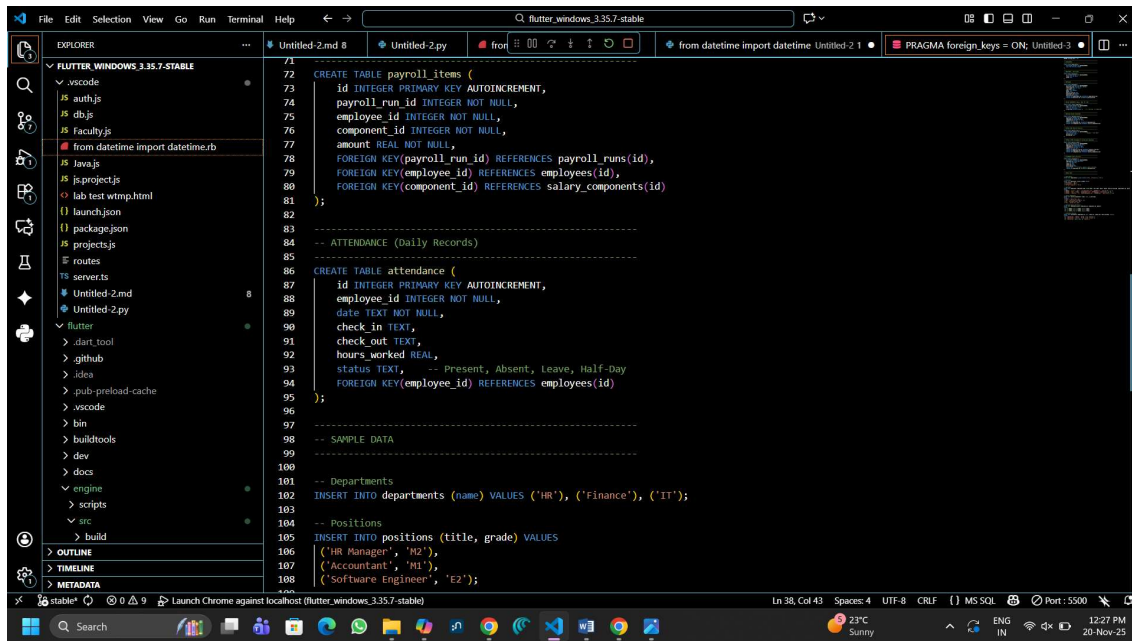
The screenshot shows a VS Code editor window with a dark theme. The Explorer panel on the left shows a file tree with folders like 'flutter_windows_3.35.7-STABLE' and 'flutter'. The main editor area displays SQL code for creating a database schema. The code includes comments for departments, positions, employees, and salary components. The status bar at the bottom indicates the file is 'Untitled-2.py' and the cursor is at line 137, column 45.

```
1 PRAGMA foreign_keys = ON;
2
3 -----
4 -- DEPARTMENTS
5
6 CREATE TABLE departments (
7     id INTEGER PRIMARY KEY AUTOINCREMENT,
8     name TEXT NOT NULL UNIQUE
9 );
10
11 -----
12 -- POSITIONS / JOB TITLES
13
14 CREATE TABLE positions (
15     id INTEGER PRIMARY KEY AUTOINCREMENT,
16     title TEXT NOT NULL,
17     grade TEXT
18 );
19
20 -----
21 -- EMPLOYEES
22
23 CREATE TABLE employees (
24     id INTEGER PRIMARY KEY AUTOINCREMENT,
25     employee_code TEXT NOT NULL UNIQUE,
26     first_name TEXT NOT NULL,
27     last_name TEXT,
28     email TEXT UNIQUE,
29     phone TEXT,
30     date_of_joining TEXT,
31     department_id INTEGER,
32     position_id INTEGER,
33     FOREIGN KEY(department_id) REFERENCES departments(id),
34     FOREIGN KEY(position_id) REFERENCES positions(id)
35 );
36
37 -----
38 -- SALARY COMPONENTS (Basic, HRA, PF, TAX)
```



The screenshot shows an IDE with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'FLUTTER WINDOWS 3.35.7-STABLE' with various files and folders. The code editor displays SQL code for creating database tables. The code includes comments for salary components, employee salary structure, payroll runs, and payroll items. The status bar at the bottom indicates the current file is 'Untitled-2.py' and the database is 'PRAGMA foreign_keys = ON;'. The system tray at the bottom shows the date and time as 12:27 PM on 20-Nov-25.

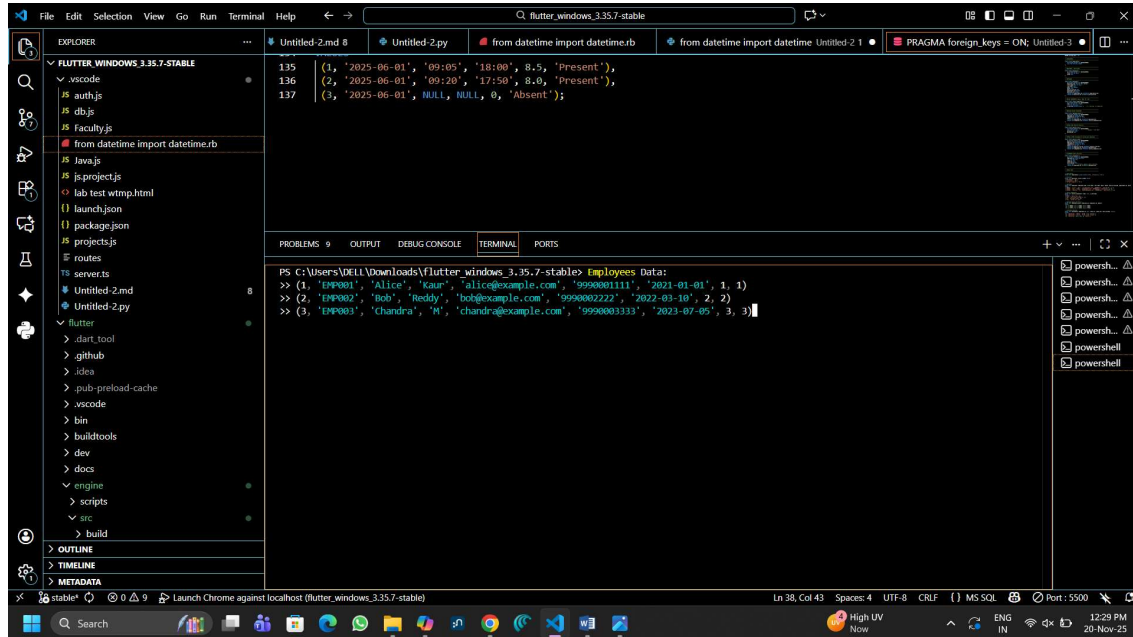
```
37 -----  
38 -- SALARY COMPONENTS (Basic, HRA, PF, TAX)  
39 -----  
40 CREATE TABLE salary_components (  
41     id INTEGER PRIMARY KEY AUTOINCREMENT,  
42     code TEXT NOT NULL UNIQUE,  
43     name TEXT NOT NULL,  
44     is_earning INTEGER DEFAULT 1 -- 1 = earning, 0 = deduction  
45 );  
46 -----  
47 -----  
48 -- EMPLOYEE SALARY STRUCTURE  
49 -----  
50 CREATE TABLE employee_salary (  
51     id INTEGER PRIMARY KEY AUTOINCREMENT,  
52     employee_id INTEGER NOT NULL,  
53     component_id INTEGER NOT NULL,  
54     amount REAL NOT NULL,  
55     FOREIGN KEY(employee_id) REFERENCES employees(id),  
56     FOREIGN KEY(component_id) REFERENCES salary_components(id)  
57 );  
58 -----  
59 -----  
60 -- PAYROLL RUN (Monthly Payroll)  
61 -----  
62 CREATE TABLE payroll_runs (  
63     id INTEGER PRIMARY KEY AUTOINCREMENT,  
64     name TEXT NOT NULL, -- Example: "June 2025"  
65     period_start TEXT,  
66     period_end TEXT  
67 );  
68 -----  
69 -----  
70 -- PAYROLL ITEMS (Breakdown of Salary per Employee)  
71 -----  
72 CREATE TABLE payroll_items (  
73     id INTEGER PRIMARY KEY AUTOINCREMENT,  
74     payroll_run_id INTEGER NOT NULL,  
75     employee_id INTEGER NOT NULL,  
76     component_id INTEGER NOT NULL,  
77     amount REAL NOT NULL,  
78     FOREIGN KEY(payroll_run_id) REFERENCES payroll_runs(id),  
79     FOREIGN KEY(employee_id) REFERENCES employees(id),  
80     FOREIGN KEY(component_id) REFERENCES salary_components(id)  
81 );  
82 -----  
83 -----  
84 -- ATTENDANCE (Daily Records)  
85 -----  
86 CREATE TABLE attendance (  
87     id INTEGER PRIMARY KEY AUTOINCREMENT,  
88     employee_id INTEGER NOT NULL,  
89     date TEXT NOT NULL,  
90     check_in TEXT,  
91     check_out TEXT,  
92     hours_worked REAL,  
93     status TEXT, -- Present, Absent, Leave, Half-Day  
94     FOREIGN KEY(employee_id) REFERENCES employees(id)  
95 );  
96 -----  
97 -----  
98 -- SAMPLE DATA  
99 -----  
100 -----  
101 -- Departments  
102 INSERT INTO departments (name) VALUES ('HR'), ('Finance'), ('IT');  
103 -----  
104 -- Positions  
105 INSERT INTO positions (title, grade) VALUES  
106     ('HR Manager', 'M2'),  
107     ('Accountant', 'M1'),  
108     ('Software Engineer', 'E2');
```



The screenshot shows an IDE with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'FLUTTER WINDOWS 3.35.7-STABLE' with various files and folders. The code editor displays SQL code for creating database tables. The code includes comments for salary components, employee salary structure, payroll runs, and payroll items. The status bar at the bottom indicates the current file is 'Untitled-2.py' and the database is 'PRAGMA foreign_keys = ON;'. The system tray at the bottom shows the date and time as 12:27 PM on 20-Nov-25.

```
71 -----  
72 -- SALARY COMPONENTS (Basic, HRA, PF, TAX)  
73 -----  
74 CREATE TABLE salary_components (  
75     id INTEGER PRIMARY KEY AUTOINCREMENT,  
76     code TEXT NOT NULL UNIQUE,  
77     name TEXT NOT NULL,  
78     is_earning INTEGER DEFAULT 1 -- 1 = earning, 0 = deduction  
79 );  
80 -----  
81 -----  
82 -- EMPLOYEE SALARY STRUCTURE  
83 -----  
84 CREATE TABLE employee_salary (  
85     id INTEGER PRIMARY KEY AUTOINCREMENT,  
86     employee_id INTEGER NOT NULL,  
87     component_id INTEGER NOT NULL,  
88     amount REAL NOT NULL,  
89     FOREIGN KEY(employee_id) REFERENCES employees(id),  
90     FOREIGN KEY(component_id) REFERENCES salary_components(id)  
91 );  
92 -----  
93 -----  
94 -- PAYROLL RUN (Monthly Payroll)  
95 -----  
96 CREATE TABLE payroll_runs (  
97     id INTEGER PRIMARY KEY AUTOINCREMENT,  
98     name TEXT NOT NULL, -- Example: "June 2025"  
99     period_start TEXT,  
100     period_end TEXT  
101 );  
102 -----  
103 -----  
104 -- PAYROLL ITEMS (Breakdown of Salary per Employee)  
105 -----  
106 CREATE TABLE payroll_items (  
107     id INTEGER PRIMARY KEY AUTOINCREMENT,  
108     payroll_run_id INTEGER NOT NULL,  
109     employee_id INTEGER NOT NULL,  
110     component_id INTEGER NOT NULL,  
111     amount REAL NOT NULL,  
112     FOREIGN KEY(payroll_run_id) REFERENCES payroll_runs(id),  
113     FOREIGN KEY(employee_id) REFERENCES employees(id),  
114     FOREIGN KEY(component_id) REFERENCES salary_components(id)  
115 );  
116 -----  
117 -----  
118 -- ATTENDANCE (Daily Records)  
119 -----  
120 CREATE TABLE attendance (  
121     id INTEGER PRIMARY KEY AUTOINCREMENT,  
122     employee_id INTEGER NOT NULL,  
123     date TEXT NOT NULL,  
124     check_in TEXT,  
125     check_out TEXT,  
126     hours_worked REAL,  
127     status TEXT, -- Present, Absent, Leave, Half-Day  
128     FOREIGN KEY(employee_id) REFERENCES employees(id)  
129 );  
130 -----  
131 -----  
132 -- SAMPLE DATA  
133 -----  
134 -----  
135 -- Departments  
136 INSERT INTO departments (name) VALUES ('HR'), ('Finance'), ('IT');  
137 -----  
138 -- Positions  
139 INSERT INTO positions (title, grade) VALUES  
140     ('HR Manager', 'M2'),  
141     ('Accountant', 'M1'),  
142     ('Software Engineer', 'E2');
```

OUTPUT:



The screenshot shows a VS Code editor with a file explorer on the left containing a project named 'FLUTTER_WINDOWS_3.35.7-STABLE'. The main editor area displays a file named 'from datetime import datetime.rb' with the following SQL code:

```
135 (1, '2025-06-01', '09:05', '18:00', 8.5, 'Present'),
136 (2, '2025-06-01', '09:20', '17:50', 8.0, 'Present'),
137 (3, '2025-06-01', NULL, NULL, 0, 'Absent');
```

The bottom panel shows the 'TERMINAL' tab with the following output:

```
PS C:\Users\DELL\Downloads\flutter_windows_3.35.7-stable> Employees Data:
>> (1, 'EMP001', 'Alice', 'Kaur', 'alice@example.com', '9990001111', '2021-01-01', 1, 1)
>> (2, 'EMP002', 'Bob', 'Reddy', 'bob@example.com', '9990002222', '2022-03-10', 2, 2)
>> (3, 'EMP003', 'Chandra', 'W', 'chandra@example.com', '9990003333', '2023-07-05', 3, 3)
```

EXPLANATION:

Core tables: departments, positions, employees, salary_components, employee_salary, payroll_runs, payroll_items, attendance.

Relations: foreign keys link employees → departments/positions; salary and payroll tables link employees ↔ salary_components.

Data: sample inserts for 3 departments, 3 positions, 3 employees, salary components, per-employee salary rows, and one-day attendance.

Notes: dates/times stored as TEXT; amounts as REAL; is_earning flags earnings vs deductions.

B) PROMPT:

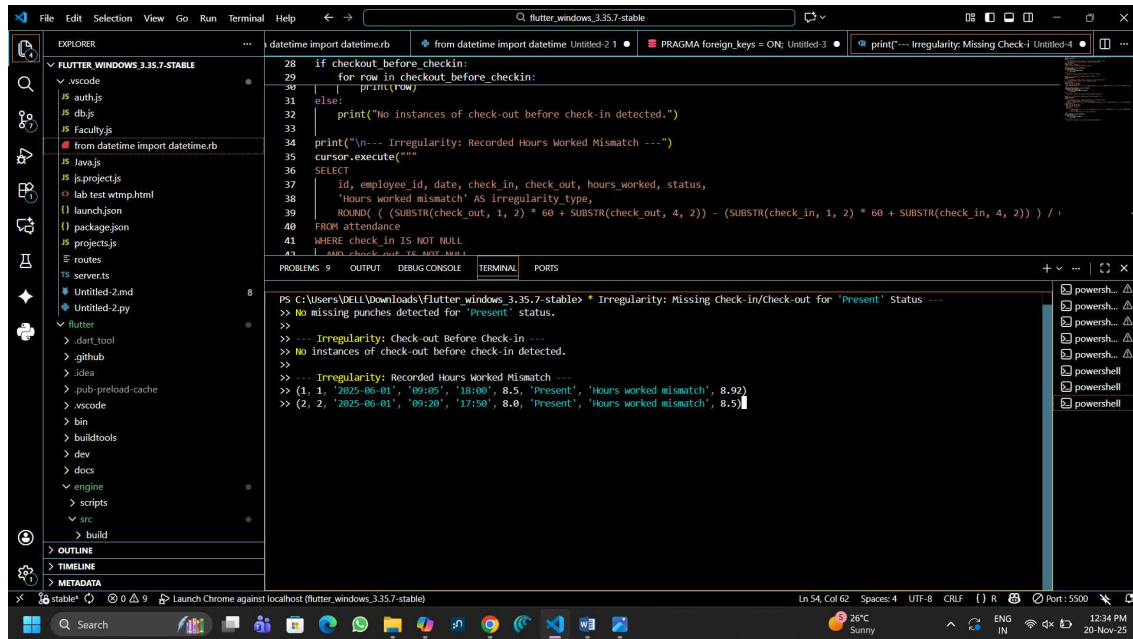
WRITE A CODE TO DESIGN A DATABASE FOR EMPLOYEE PAYROLL AND ATTENDANCE IN SQL.

CODE:

```
1 print("---- Irregularity: Missing Check-in/Check-out for 'Present' Status ----")
2 cursor.execute("""
3 SELECT
4     id, employee_id, date, check_in, check_out, hours_worked, status,
5     'Missing punch for present status' AS irregularity_type
6 FROM attendance
7 WHERE status = 'Present'
8     AND (check_in IS NULL OR check_out IS NULL);
9 """)
10 missing_punches = cursor.fetchall()
11 if missing_punches:
12     for row in missing_punches:
13         print(row)
14 else:
15     print("no missing punches detected for 'Present' status.")
16
17 print("\n--- Irregularity: Check-out Before Check-in ---")
18 cursor.execute("""
19 SELECT
20     id, employee_id, date, check_in, check_out, hours_worked, status,
21     'check-out before check-in' AS irregularity_type
22 FROM attendance
23 WHERE check_in IS NOT NULL
24     AND check_out IS NOT NULL
25     AND (SUBSTR(check_out, 1, 2) * 60 + SUBSTR(check_out, 4, 2)) < (SUBSTR(check_in, 1, 2) * 60 + SUBSTR(check_in, 4, 2));
26 """)
27 checkout_before_checkin = cursor.fetchall()
28 if checkout_before_checkin:
29     for row in checkout_before_checkin:
30         print(row)
31 else:
32     print("no instances of check-out before check-in detected.")
33
34 print("\n--- Irregularity: Recorded Hours Worked Mismatch ---")
35 cursor.execute("""
36 SELECT
37     id, employee_id, date, check_in, check_out, hours_worked, status,
38     'hours worked mismatch' AS irregularity_type
39 """)
40 hours_worked_mismatch = cursor.fetchall()
41 if hours_worked_mismatch:
42     for row in hours_worked_mismatch:
43         print(row)
44 else:
45     print("no significant hours worked mismatches detected.")
```

```
28 if checkout_before_checkin:
29     for row in checkout_before_checkin:
30         print(row)
31 else:
32     print("no instances of check-out before check-in detected.")
33
34 print("\n--- Irregularity: Recorded Hours Worked Mismatch ---")
35 cursor.execute("""
36 SELECT
37     id, employee_id, date, check_in, check_out, hours_worked, status,
38     'Hours worked mismatch' AS irregularity_type,
39     ROUND( ( (SUBSTR(check_out, 1, 2) * 60 + SUBSTR(check_out, 4, 2)) - (SUBSTR(check_in, 1, 2) * 60 + SUBSTR(check_in, 4, 2)) ) / 60,
40     FROM attendance
41 WHERE check_in IS NOT NULL
42     AND check_out IS NOT NULL
43     AND status = 'Present'
44     AND ABS(
45         ( (SUBSTR(check_out, 1, 2) * 60 + SUBSTR(check_out, 4, 2)) - (SUBSTR(check_in, 1, 2) * 60 + SUBSTR(check_in, 4, 2)) ) / 60,
46         - hours_worked
47     ) > 0.1; -- Tolerance of 0.1 hours (6 minutes)
48 """)
49 hours_mismatch = cursor.fetchall()
50 if hours_mismatch:
51     for row in hours_mismatch:
52         print(row)
53 else:
54     print("no significant hours worked mismatches detected.")
```

OUTPUT:



The screenshot shows a VS Code editor with a file named `from datetime import datetime.rb` open. The editor contains a SQL query that checks for three irregularities: missing punches, reversed times, and hours mismatch. The query is executed, and the results are displayed in the terminal window at the bottom.

```
28 If checkout before checkin:
29   for row in checkout_before_checkin:
30     print(row)
31   else:
32     print("No instances of check-out before check-in detected.")
33
34 print("\n--- Irregularity: Recorded Hours Worked Mismatch ---")
35 cursor.execute("""
36 SELECT
37   id, employee_id, date, check_in, check_out, hours_worked, status,
38   'Hours worked mismatch' AS irregularity_type,
39   ROUND( ( (SUBSTR(check_out, 1, 2) * 60 + SUBSTR(check_out, 4, 2)) - (SUBSTR(check_in, 1, 2) * 60 + SUBSTR(check_in, 4, 2)) ) / :
40 FROM attendance
41 WHERE check_in IS NOT NULL
42 AND check_out IS NOT NULL
43 """)
44
45 # Print the results
46 for row in cursor.fetchall():
47   print(row)
```

The terminal output shows the results of the query execution:

```
PS C:\Users\DELL\Downloads\Flutter_windows_3.35.7-stable> * Irregularity: Missing check-in/check-out for 'Present' Status ---
>> No missing punches detected for 'Present' status.
>>
>> --- Irregularity: Check-out Before Check-in ---
>> No instances of check-out before check-in detected.
>>
>> --- Irregularity: Recorded Hours Worked Mismatch ---
>> (1, 1, '2025-06-01', '09:05', '18:00', 8.5, 'Present', 'Hours worked mismatch', 8.92)
>> (2, 2, '2025-06-01', '09:20', '17:50', 8.0, 'Present', 'Hours worked mismatch', 8.5)
```

EXPLANATION:

File: b.sql — short summary: scans attendance for three irregularities and prints results.

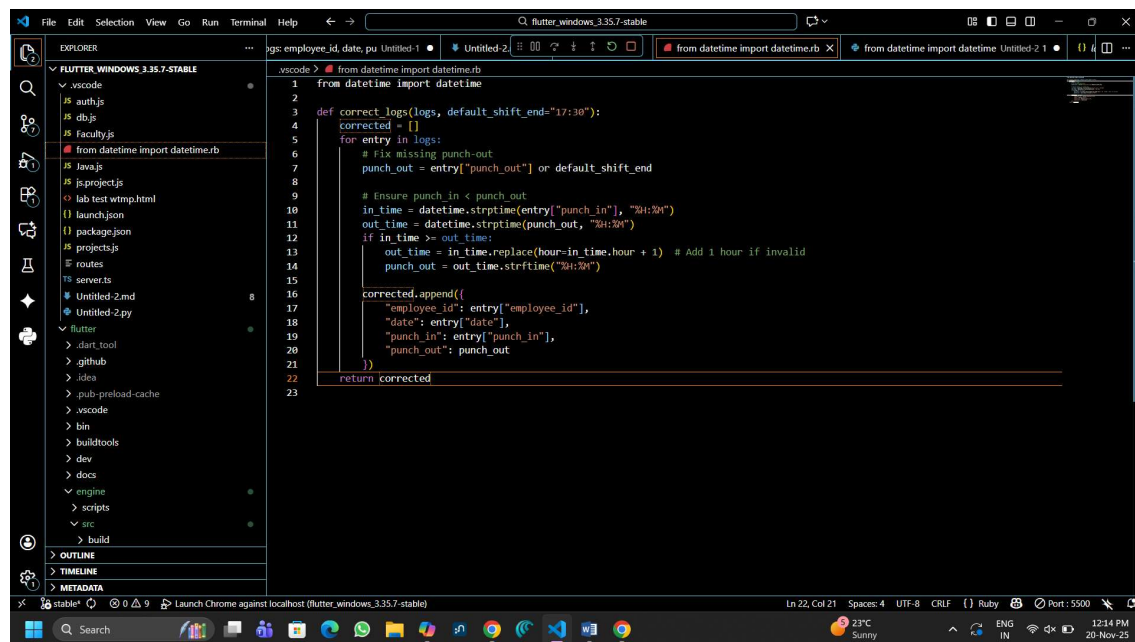
Checks: Missing punch: Present but missing check-in/out; Reversed times: check-out earlier than check-in; Hours mismatch: calculated hours (from HH:MM) differ from hours_worked by >0.1 hrs.

Q2:

A) PROMPT:

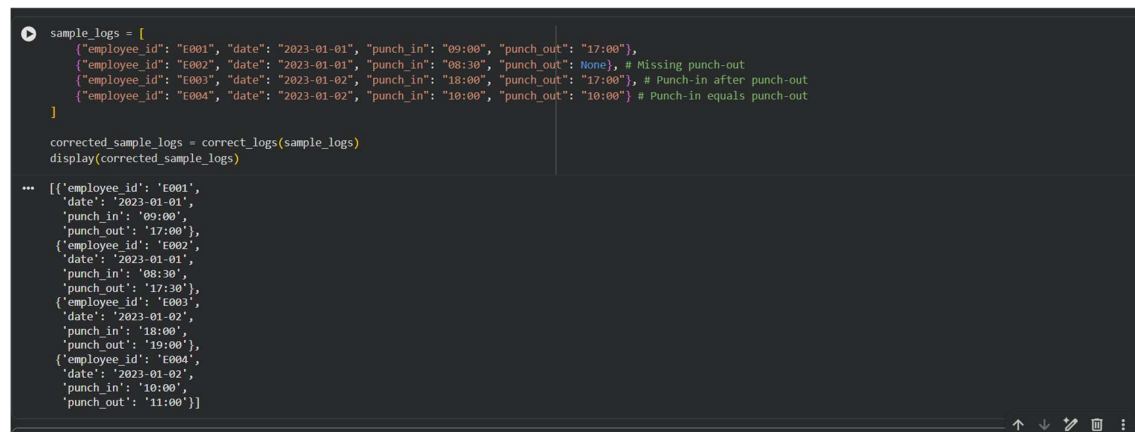
Create a function that corrects punch-in/punch-out logs by handling missing punch-outs, fixing out-of-order timestamps, and removing overlaps using AI-based assumptions like default shift end time.

CODE:



```
1 from datetime import datetime
2
3 def correct_logs(logs, default_shift_end="17:30"):
4     corrected = []
5     for entry in logs:
6         # Fix missing punch-out
7         punch_out = entry["punch_out"] or default_shift_end
8
9         # Ensure punch_in < punch_out
10        in_time = datetime.strptime(entry["punch_in"], "%H:%M")
11        out_time = datetime.strptime(punch_out, "%H:%M")
12        if in_time >= out_time:
13            out_time = in_time.replace(hour=in_time.hour + 1) # Add 1 hour if invalid
14            punch_out = out_time.strftime("%H:%M")
15
16        corrected.append({
17            "employee_id": entry["employee_id"],
18            "date": entry["date"],
19            "punch_in": entry["punch_in"],
20            "punch_out": punch_out
21        })
22    return corrected
23
```

OUTPUT:



```
sample_logs = [
    {"employee_id": "E001", "date": "2023-01-01", "punch_in": "09:00", "punch_out": "17:00"},
    {"employee_id": "E002", "date": "2023-01-01", "punch_in": "08:30", "punch_out": None}, # Missing punch-out
    {"employee_id": "E003", "date": "2023-01-02", "punch_in": "18:00", "punch_out": "17:00"}, # Punch-in after punch-out
    {"employee_id": "E004", "date": "2023-01-02", "punch_in": "10:00", "punch_out": "10:00"} # Punch-in equals punch-out
]

corrected_sample_logs = correct_logs(sample_logs)
display(corrected_sample_logs)

[[{"employee_id": "E001",
  "date": "2023-01-01",
  "punch_in": "09:00",
  "punch_out": "17:00"},
 {"employee_id": "E002",
  "date": "2023-01-01",
  "punch_in": "08:30",
  "punch_out": "17:30"},
 {"employee_id": "E003",
  "date": "2023-01-02",
  "punch_in": "18:00",
  "punch_out": "19:00"},
 {"employee_id": "E004",
  "date": "2023-01-02",
  "punch_in": "10:00",
  "punch_out": "11:00"}]]
```

EXPLANATION:

Missing punch-out → filled with default "17:30".

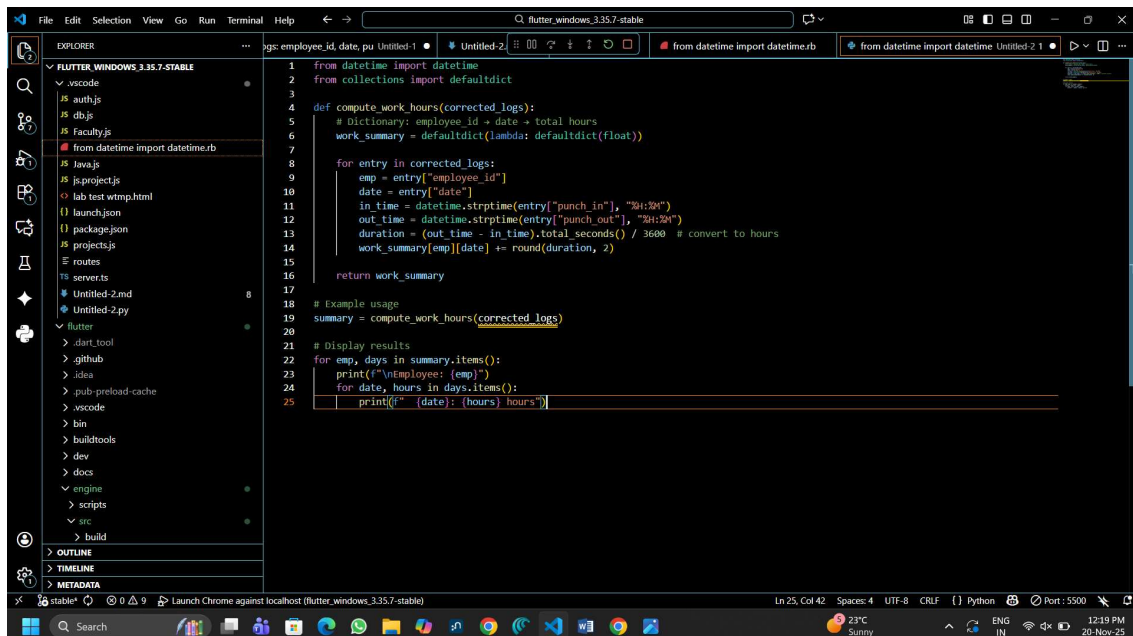
Out-of-order times → corrected by adding 1 hour.

Returns clean log entries ready for further processing.

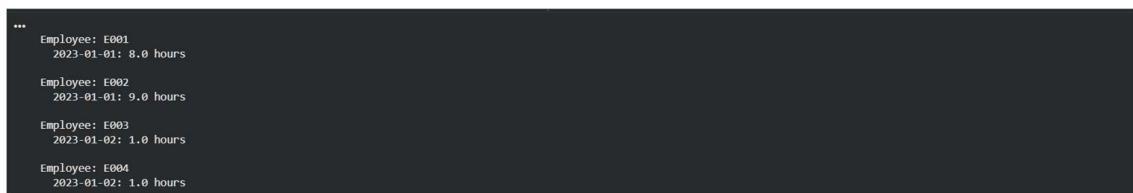
B) PROMPT:

Implement a function that takes corrected punch-in/punch-out logs and calculates total working hours per employee per day. The function should support multiple sessions per day and return a structured summary showing each employee's daily total hours.

CODE:

A screenshot of a Visual Studio Code editor window. The Explorer sidebar on the left shows a file tree with folders like .vscode, .dart_tool, and .github, and files like launch.json, package.json, routes, servers, and build. The main editor area displays a Python script named 'compute_work_hours.py'. The script imports datetime and defaultdict, defines a function 'compute_work_hours' that takes 'corrected_logs' and returns a summary dictionary, and includes an example usage section. The script uses datetime.strptime to parse punch-in and punch-out times, calculates the duration in seconds, and rounds it to the nearest hour. The status bar at the bottom shows 'Ln 25, Col 42', 'Spaces: 4', 'UTF-8', 'CRLF', and 'Python'.

OUTPUT:

A screenshot of a terminal window showing the output of the Python script. The output displays the total working hours for four employees (E001, E002, E003, E004) on two dates (2023-01-01 and 2023-01-02). The results are: Employee: E001, 2023-01-01: 8.0 hours; Employee: E002, 2023-01-01: 9.0 hours; Employee: E003, 2023-01-02: 1.0 hours; Employee: E004, 2023-01-02: 1.0 hours.

EXPLANATION:

Uses `datetime.strptime()` to convert time strings to time objects.

Calculates duration in hours for each session.

Aggregates hours per employee per day using nested dictionaries.

Supports multiple sessions on the same day.